



# DIARY: Differentially Private Recovery with Adaptive Privacy Budgets in Federated Unlearning

Hengzhi Wang  
Shenzhen University  
College of Computer Science and  
Software Engineering  
Shenzhen, China  
whz@szu.edu.cn

Lu Dai  
Shenzhen University  
College of Computer Science and  
Software Engineering  
Shenzhen, China  
2500101038@mails.szu.edu.cn

Xianliang Zhang  
Shenzhen University  
College of Computer Science and  
Software Engineering  
Shenzhen, China  
2500101012@mails.szu.edu.cn

Haoran Chen  
Shenzhen University  
College of Computer Science and  
Software Engineering  
Shenzhen, China  
2400101062@mails.szu.edu.cn

Juncheng Hu\*  
Jilin University  
College of Computer Science and  
Technology  
Changchun, China  
jchu@jlu.edu.cn

Kun Yang  
University of Essex  
School of Computer Science and  
Electronic Engineering  
Colchester, United Kingdom  
kunyang@essex.ac.uk

## Abstract

Federated Unlearning (FU) has emerged as a promising paradigm for effectively removing the influence of specific data of clients from the global model in federated learning. It can further enhance personal data privacy for individual clients and eliminate the impact of malicious attacks like poisoning. Due to these benefits, many FU methods have been analyzed and proposed. Yet, they largely overlook external threats against FU systems, such as gradient inversion attacks that reconstruct client data from shared gradients, posing serious privacy risks to other participating clients. Motivated by this, we propose DIARY, a Differential privacy IntegrAted federated recovery framework to address these dual threats. DIARY presents a privacy budget allocation method, whose insight lies in adaptively assigning appropriate privacy budgets to various training and recovery phases to fully utilize the global privacy budget of each client, balancing the trade-off between privacy and utility. Furthermore, DIARY introduces a novel Federated noise-Immune aNomaly Detection (FIND) module. The deep integration of FIND with two-level selective storage and model rollback mechanisms contributes to model recovery, while significantly reducing the associated overhead. Finally, both rigorous theoretical analysis and extensive simulations compared with state-of-the-art methods are conducted to validate the effectiveness of DIARY.

## CCS Concepts

• **Security and privacy** → *Web application security*; • **Computing methodologies** → *Distributed algorithms*.

## Keywords

Federated unlearning, differential privacy, model recovery

\*The corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2307-0/2026/04  
<https://doi.org/10.1145/3774904.3792423>

## ACM Reference Format:

Hengzhi Wang, Lu Dai, Xianliang Zhang, Haoran Chen, Juncheng Hu, and Kun Yang. 2026. DIARY: Differentially Private Recovery with Adaptive Privacy Budgets in Federated Unlearning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792423>

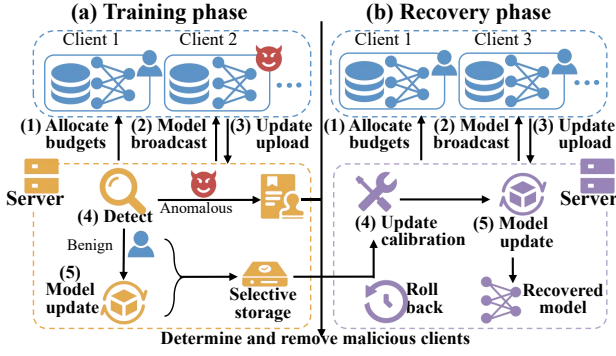
## Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18321980>.

## 1 Introduction

Federated Unlearning (FU) has emerged as a promising solution to erase the impact of specific clients' data on the global model in Federated Learning (FL). Its primary objective is to make the global model appear as if it has never seen the unlearned data. The development of federated unlearning is driven by the increasing demand for data protection and privacy preservation. For example, the European Union's General Data Protection Regulation (GDPR) [35] and the previous "Right to Be Forgotten" [9] entails the right for people to withdraw their consent to any processing operation on their data in certain contexts. Similar statements can also be found in Canada's Consumer Privacy Protection Act (CPPA) [18] and California's Consumer Privacy Act (CCPA) [13].

In practice, federated unlearning can not only protect personal data privacy, but also be utilized to eliminate the impact of poisoning attacks [2, 3, 11, 38], such as backdoor attacks [2, 38]. Specifically, an unlearning operation can be performed on detected malicious clients, a process also referred to as model recovery [12, 26]. The most straightforward recovery method involves excluding the malicious clients and retraining the model from scratch using the remaining benign clients, but this method typically incurs significant computational and communication overhead. State-of-the-art methods (e.g., FedRecover [7], FedEraser [26], Crab [15]) are primarily based on traditional FL frameworks and focus on reducing recovery overhead by storing historical global models and client updates, which are utilized during the recovery phase. Methods SISA [4], FedCIO [33], and FATS [34] train multiple sub-models via



**Figure 1: An illustration of the framework of DIARY.**

client sharding and data partitioning, enabling recovery of a subset of sub-models during the recovery phase and improving efficiency.

However, these methods mainly focus on eliminating the impact of internal attacks but overlook the threats posed by external attacks (e.g., gradient inversion attacks [10, 39, 41]) to the privacy of other clients. FedADP [16] attempted to address this gap by introducing Differential Privacy (DP) [8] into federated unlearning. Nevertheless, its privacy budget allocation strategy is relatively simplistic and fails to adequately consider the impact of noise injection on the model recovery process. Liu et al. [28] employed secure aggregation to protect client data privacy in federated unlearning, but this incurs extra communication and computation costs. There remains a lack of a model recovery method in federated unlearning that balances privacy, recovery performance, and overhead.

Achieving this goal presents several challenges: first, designing a sophisticated, adaptive privacy budget allocation mechanism is necessary to protect client data privacy while promoting the model recovery process. The key challenge lies in proposing a precise and compatible privacy accounting method to compute the optimal composition of privacy loss given sequential varying DP algorithms over training and recovery phases. Second, the introduction of DP noise significantly increases the difficulty of malicious client detection, resulting in traditional FL detection algorithms failing in our settings. The key challenge lies in the conflict between privacy preservation and detection efficiency. Third, it is essential and challenging to conduct rigorous theoretical analysis on privacy guarantees, specifically coupled with the adaptive privacy budget allocation for the entire federated unlearning process.

To overcome these challenges, we propose DIARY, a **Differential Privacy integrAted federated recoveryY** framework, whose brief illustration is depicted in Fig.1. To solve the first challenge, DIARY employs an adaptive, dynamic privacy budget allocation mechanism, as well as a precise privacy accounting method. The insight is to dynamically and adaptively distribute appropriate privacy budgets in a noise injection way to various training and recovery phases for fully utilizing the global privacy budget of each client. This provably provides privacy guarantees while actively facilitating the model recovery process, thereby enhancing the performance of model recovery. For the second challenge, DIARY further introduces a novel **Federated noise-Immune aNomaly Detection (FIND)** module, which identifies the final set of malicious clients and leverages its round-by-round detection capability to provide support for the recovery process. In addition, DIARY incorporates

an importance-based two-level selection mechanism to store critical historical information, effectively reducing storage and computational overhead while maintaining the performance of the recovered model. For the third challenge, we conduct a comprehensive theoretical analysis on DIARY, providing privacy guarantees of the privacy accounting method. Finally, we conduct comprehensive experiments on three standard datasets and compare against multiple state-of-the-art methods. The results validate the superiority of our privacy budget allocation mechanism and demonstrate the effectiveness and efficiency of DIARY.

Our main contributions are summarized as follows:

- **Problem.** To the best of our knowledge, DIARY is the first framework to address the adaptive and personalized privacy budget allocation problem in federated unlearning. It includes a precise privacy accounting method to derive optimal composition for sequential DP algorithms over rounds.
- **Method.** We propose DIARY, a differential privacy integrated federated recovery framework that effectively balances the trade-off between privacy protection and recovery performance while significantly reducing recovery overhead.
- **Analysis.** We provide rigorous theoretical analysis of DIARY, including its privacy budget allocation mechanism and privacy accounting method.
- **Simulation.** We conduct extensive simulations based on standard datasets and compare with state-of-the-art methods. The results demonstrate the effectiveness of DIARY.

The rest of the paper is organized as follows. We review related work in Section 2, introduce the preliminaries in Section 3. We design our framework in Section 4. The framework is evaluated in Section 5 theoretically and in Section 6 numerically. The paper is concluded in Section 7. Appendices are in Section A.

## 2 Related Work

### 2.1 Federated Unlearning

Recent research has focused on reducing the overhead of the recovery process while maintaining model performance, which primarily falls into two categories: Methods such as SISA [4], FedCIO [33], FATS [34] and Lin et al. [25] train multiple sub-models via client sharding and data partitioning, enabling recovery of only a subset of sub-models during the recovery phase. However, these methods become inefficient and costly when malicious clients are numerous and distributed across multiple shards. The second category involves historical information utilization. FedRecover [7], FedEraser [26], and Crab [15] leverage historical updates or models to recover. Critically, existing approaches suffer from two major limitations. First, they lack deep integration with detection algorithms, employing them merely to identify malicious clients rather than optimizing the recovery. Second, they overlook external privacy threats such as gradient inversion attacks. Although recent works [16, 28] attempt to mitigate this via Differential Privacy or secure aggregation, they are hindered by either simplistic budget allocation or prohibitive computational overheads.

### 2.2 Differential Privacy in Federated Learning

While FL mitigates privacy risks by localizing raw data, model gradients remain susceptible to leakage data information [10, 41]. To

address this, DP [8] was introduced into deep learning via DP-SGD [1], serving as a formal mathematical framework to quantify and limit privacy loss. However, the noise injection required by DP creates an inherent trade-off between privacy and model utility. Consequently, significant research has focused on tighter privacy accounting to improve this trade-off. For instance, Abadi et al. [1] proposed the Moments Accountant to reduce privacy cost estimates compared to Strong Composition [8], and Mironov et al. [29] enhanced efficiency using Rényi Differential Privacy (RDP).

In the context of FL, Wei et al. [36] established a convergence-guaranteed DP-FL framework, while Liu et al. [27] introduced rPDP-FL, which leveraged RDP and two-phase hybrid sampling for improved privacy amplification. Despite these advancements, most existing approaches adopt a uniform privacy budget allocation across all iterations. This strategy is arguably suboptimal. Intuitively, the optimization process tolerates lower precision in early phases but demands accurate updates as parameters converge. Although DP-Flames [14] and FL-DPCR [5] investigate adaptive privacy budget allocation, they do not support personalized allocation, incurring significant computational and storage overhead.

### 3 Preliminaries

#### 3.1 Federated Learning

We consider a general federated learning system comprising one server and  $N$  clients, represented by the set  $[N]$ . The primary objective of this system is to collaboratively learn a model under the constraint that raw client data remains decentralized. The server maintains a global model parameterized by the vector  $\omega_t$ , where  $t \in [T]$  indexes the communication round in a total of  $T$  discrete rounds. The number of local training epochs at each round is  $E$ . Each client  $i \in [N]$  maintains a local model  $\omega_{i,t}$  and a local dataset  $D_i$ . The loss of client  $i$  on a data point  $d = (x, y) \in D_i$  is denoted as  $f(\omega_{i,t}, d)$ . The average loss over the local dataset  $D_i$  is given by:

$$F(\omega_{i,t}, D_i) = 1/|D_i| \sum_{d \in D_i} f(\omega_{i,t}, d) \quad (1)$$

The local objective of client  $i$  is to find a model  $\omega_{i,t}$  that achieves a sufficiently small average loss:

$$\omega_{i,t} = \arg \min_{\omega} F(\omega_{i,t}, D_i) \quad (2)$$

The client then computes the update  $g_{i,t} = \omega_{i,t} - \omega_t$  and sends it to the server. The server aggregates all updates received in round  $t$  with an aggregation rule  $\mathcal{A}$  (e.g. FedAvg):

$$g_t = \mathcal{A}(g_{1,t}, g_{2,t}, \dots, g_{i,t}) = 1/N \sum_{i=1}^N g_{i,t} \quad (3)$$

and updates the global model accordingly  $\omega_{t+1} = \omega_t + \eta g_t$ . Through iterative repetition of this process, the server eventually obtains a converged global model.

#### 3.2 Threat Model

Due to its distributed and collaborative architecture, federated learning exhibits multiple security vulnerabilities and are susceptible to attacks from various sources. These threats can be broadly categorized into two main types: internal attacks and external attacks.

**External threat: gradient inversion attack.** This attack aims to obtain sensitive clients' data. The adversary first obtains the victim client's gradient  $g$  and the global model parameters  $\omega$  through eavesdropping. The adversary then constructs a set of dummy

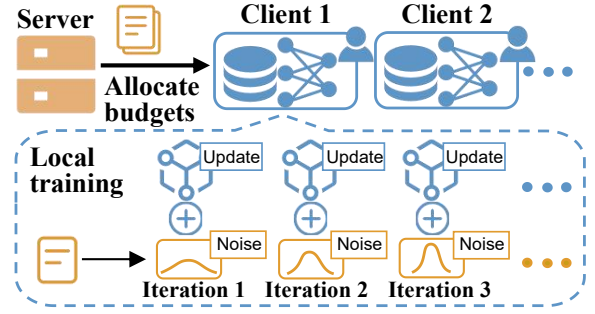


Figure 2: An illustration of privacy budget allocation.

data  $d' = (x', y')$  and performs forward and backward propagation based on the current global model to generate corresponding dummy gradients. By iteratively optimizing the dummy data, the attacker minimizes the distance between the dummy gradients and the victim's true gradients:

$$\arg \min_{d'} \|\nabla f(\omega, d') - g\|^2 \quad (4)$$

After multiple iterations, the attacker obtains fabricated samples that closely approximate the victim's private data, i.e.  $d' \approx d$ .

**Internal threat: backdoor attack.** Backdoor attack aims to alter the behavior of the model on some specific examples with no negative impact on the benign performance. Formally, the adversary manipulates the global model  $\omega$  to the poisoned model  $\omega'$ . For any inputs  $x$ , we have the outputs below:

$$\omega'(x) = \begin{cases} y' \neq \omega(x) & \forall x \in D_A \\ \omega(x) & \forall x \notin D_A \end{cases} \quad (5)$$

where  $D_A$  denotes the poisoned dataset of adversary, and  $y'$  denotes the specific label modified by the adversary of the input  $x$ .

#### 3.3 Differential Privacy

**DEFINITION 1** ( $(\epsilon, \delta)$ -DP [8]). A randomized mechanism  $\mathcal{M} : \mathbb{D} \rightarrow \mathbb{R}$  with domain  $\mathbb{D}$  and range  $\mathbb{R}$  satisfies  $(\epsilon, \delta)$ -DP, if for all measurable sets  $S \subseteq \mathbb{R}$  and for any two adjacent databases  $D, D' \in \mathbb{D}$  that differ by only one record, i.e.,

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D') \in S] + \delta \quad (6)$$

where  $\epsilon > 0$  is the privacy budget that quantifies the level of privacy protection provided by  $\mathcal{M}$ . The relaxation rate  $\delta$  enables the firm  $\epsilon$ -DP to be broken with probability  $\delta$ .

Following previous studies [8], we establish the privacy-preserving mechanism  $\mathcal{M}$  based on Gaussian distributions, i.e.,

$$\tilde{g} = \text{clip}(g, \Delta_s) + \mathcal{N}(0, \Delta_s^2 \sigma^2 I) \quad (7)$$

where  $\mathcal{N}(0, \Delta_s^2 \sigma^2 I)$  is a normal distribution with a mean of 0 and a standard deviation of  $\Delta_s \sigma$ .  $\sigma$  is the noise multiplier.  $\text{clip}(g, \Delta_s)$  is the function that clips the L2 norm of  $g$  to  $\Delta_s$ , and  $\Delta_s$  denotes the sensitivity of Eq. 2 essentially reflecting the maximum absolute distance between any two models, i.e.,

$$\Delta_s = \max_{D, D'} \|\arg \min_{\omega} F(\omega, D) - \arg \min_{\omega} F(\omega, D')\|. \quad (8)$$

### 4 Design of DIARY

Our objective is to design an efficient recovery method with the following properties: (1) **Effectiveness**: It must eliminate the impact of malicious updates on the global model. (2) **Performance**: Benign

**Algorithm 1: Privacy Cost Estimation (PCE)**


---

**Input:** Total number of iterations  $TE$ , initial noise multipliers  $\sigma_0$ , relaxation rate  $\delta$ , data sampling rate  $q$ , decay rate  $\gamma$ , global parameters  $\delta^*$  and  $\xi$

**Output:** Global privacy loss  $\varepsilon^*$  after  $TE$  iterations

```

1 foreach iteration  $\tau \in [TE]$  do
2    $\sigma_\tau \leftarrow \sigma_0 / (1 + \tau)^\gamma$ ;
3    $\varepsilon_\tau \leftarrow \sqrt{2 \log(1.25/\delta)} / \sigma_\tau$ ;
4    $\varepsilon_\tau \leftarrow \log(1 + q(e^{\varepsilon_\tau} - 1))$ ;
5  $\delta = q\delta$ ;  $\bar{\varepsilon} \leftarrow \sum_{\tau \in [TE]} \varepsilon_\tau / TE$ ;
6  $\theta \leftarrow \xi / (TE \cdot (1 + \bar{\varepsilon}) + 1)$ ;  $\tilde{\varepsilon} \leftarrow \ln(1 + \theta)$ ;
7 foreach iteration  $\tau \in [TE]$  do
8    $a_\tau \leftarrow \lceil \varepsilon_\tau \cdot (1/\theta + 1) \rceil$ ;  $\hat{\varepsilon}_\tau \leftarrow a_\tau \cdot \tilde{\varepsilon}$ ;
9 Find the minimum integer  $a^* \in [\sum_{\tau \in [TE]} a_\tau]$  based on
   binary search, such that Eq. 9 holds;
10 return Global privacy loss  $\varepsilon^* = a^* \cdot \tilde{\varepsilon}$ 

```

---

performance of the global model must be preserved to maintain its utility. (3) **Efficiency:** It can rapidly restore the global model with low overhead. (4) **Privacy:** It is required to guarantee data privacy for all participating benign clients. The overall DIARY is shown in Fig. 1. The detailed description of DIARY and a comprehensive list of notations used in this paper are provided in the Appendix A.2.

#### 4.1 Privacy Budget Allocation

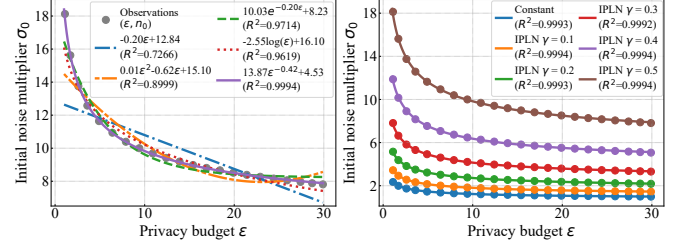
We explore how to allocate the privacy budget to each training iteration for each client, that is, determining the noise multiplier  $\sigma_\tau$ . Fig. 2 shows the process of privacy budget allocation.

**Dynamic noise injection.** Inspired by [21], the optimization process tolerates lower precision in early phases but demands accurate updates as parameters converge. Therefore, DIARY adopts the Inverse Power Law Noise (IPLN) strategy to dynamically adjust the noise multiplier  $\sigma_\tau$ . Specifically,  $\sigma_\tau$  decays with the iteration count  $\tau$  according to  $\sigma_\tau = \sigma_0 / (1 + \tau)^\gamma$ , where the hyperparameter  $\gamma$  controls the decay rate. The superiority of IPLN is empirically validated in Section 6.

**Privacy cost estimation.** To more accurately estimate the total privacy cost under dynamic noise injection, we propose a novel privacy accounting method. This method first employs privacy amplification with Poisson sampling to bound the privacy cost of a single iteration and subsequently applies the composition method as shown in Eq. 9 to compute the global privacy cost/loss (see Algorithm 1 for details). The privacy cost upper bound derived by our method is tighter than other composition theorems, with the complete theoretical proof provided in Section 5.

$$\frac{\sum_{S \subseteq [TE]} \max \{e^{\sum_{\tau \in S} \hat{\varepsilon}_\tau} - e^{a^* \tilde{\varepsilon} + \sum_{\tau \notin S} \hat{\varepsilon}_\tau}, 0\}}{\prod_{\tau=1}^{TE} (1 + e^{\hat{\varepsilon}_\tau})} \leq 1 - \frac{1 - \delta^*}{(1 - \delta)^{TE}}. \quad (9)$$

**CurveFitting.** Given the total privacy budget  $\varepsilon$  and the total number of iterations, the initial noise multiplier  $\sigma_0$  can be determined using Algorithm 1 combined with a binary search, ensuring that the total privacy budget  $\varepsilon$  is nearly exhausted without being exceeded by the end of training (e.g.  $\varepsilon \approx \varepsilon^*$ ). However, in some practical scenarios involving thousands of clients with different privacy budgets, this approach would impose a significant computational



**Figure 3: Initial noise multiplier  $\sigma_0$  for IPLN with respect to privacy budget  $\varepsilon$  with parameters  $T = 50, E = 2, q = 0.01, \gamma = 0.5$  (left part) and under various  $\gamma$  values (right part).**

burden on the server. To address this, we employ a CurveFitting method to efficiently determine the initial noise multiplier  $\sigma_0$  for each client based on their personalized privacy budgets. As shown in Fig. 3a, the selected best-fit function, which achieves a coefficient of determination ( $R^2$ ) as high as 99.9%, has a simple form:

$$\sigma_0 \approx \phi_1 \varepsilon^{\phi_2} + \phi_3. \quad (10)$$

In Fig. 3b, we illustrate the relationship between the initial noise multiplier  $\sigma_0$  and the privacy budget  $\varepsilon$  under various decay rates  $\gamma$  of IPLN. This reveals a core trade-off between the initial noise multiplier  $\sigma_0$  and the decay rate  $\gamma$ . Specifically, if the decay rate  $\gamma$  is too small, IPLN approaches a constant noise addition, which diminishes its benefit to the training process. Conversely, if the decay rate  $\gamma$  is too large, the excessively high noise multiplier in early training phase may prevent the model from learning meaningful features, leading to diminishing training performance.

#### 4.2 Design of FIND

Our goal is to design a detection method that not only identifies malicious clients but also provides substantial support for subsequent recovery phase. Due to the perturbation introduced by DP noise on client updates, the anomalous statistical features relied upon by conventional detection methods may no longer hold. Inspired by Robust-DPFL [32], FIND employs element-wise aggregation of client updates to mitigate the impact of noise based on Eq. 11. It then computes the sum of distances for each aggregated result  $Z_i$  relative to others to enhance outlier distinctiveness [22, 24]. Finally, using dynamic HDBSCAN [6] clustering algorithm, it adaptively identifies and excludes malicious updates from the current round of aggregation. Specifically, assuming a benign majority, we set the minimum cluster size to  $N/2 + 1$  (a more conservative value can also be adopted based on requirements). Compared to other clustering methods [23], dynamic HDBSCAN minimizes misjudgment rates and demonstrates robustness against sophisticated hybrid attacks, such as simultaneous random perturbation and stealthy poisoning [31]. The pseudocode of FIND is presented in Algorithm 2.

$$Z_i = \mathcal{Z}(\tilde{g}_{i,t}) = \mathcal{Z}(\text{clip}(g_{i,t}, \Delta_s) + \mathcal{N}(0, \Delta_s^2 \sigma^2 I)) \approx 1/d \sum_{j=1}^d g_{i,t}[j]. \quad (11)$$

#### 4.3 Selective Information Storage

We adopt two-level selective historical information storage strategy designed to reduce storage costs while preserving the performance of the recovered global model. Detailed procedures are provided in Algorithm 3 and Algorithm 4 in the Appendix A.2.

**Algorithm 2: FIND**

**Input:** The model updates of clients  $\{\tilde{g}_{0,t}, \tilde{g}_{1,t}, \dots, \tilde{g}_{N-1,t}\}$  at the  $t$ -th round

**Output:** Benign clients set  $S_{b,t}$ , Malicious clients set  $S_{m,t}$

```

1 for  $i \leftarrow 0$  to  $N - 1$  do
2    $Z_i \leftarrow \mathcal{Z}(\tilde{g}_{i,t})$  based on Eq. 11;
3 for  $i, j \in \{0, 1, \dots, N - 1\}$  do
4    $Z'_i \leftarrow \sum_{j,j \neq i}^N |Z_i - Z_j|$ 
5  $S_{b,t}, S_{m,t} \leftarrow \text{HDBSCAN}(Z'_0, Z'_1, \dots, Z'_{N-1})$ ;
6 return  $S_{b,t}, S_{m,t}$ 

```

**Round selection.** We premise that significant model parameter shifts and higher privacy costs signal critical learning phases with high information value. Accordingly, we define a round's importance score as the weighted sum of the Kullback-Leibler (KL) divergence between adjacent global models and the privacy cost, balanced by hyperparameter  $\zeta$ , as presented in Eq. 12. The server buffers historical information within a dynamic time window. Upon triggering a checkpoint, determined by global loss reduction and total privacy budget utilization, we persist only the top  $\lambda$  fraction of rounds with the highest scores and reset the time window.

$$P_t = (1 - \zeta) \cdot \mathcal{D}_{kl}(\omega_t, \omega_{t+1}) + \zeta \cdot \varepsilon_t, \quad (12)$$

$$\mathcal{D}_{kl}(\omega_t, \omega_{t+1}) = \sum p(\omega_{t+1}) \cdot \log \frac{p(\omega_{t+1})}{p(\omega_t)}.$$

where  $p(\omega_t)$  denotes the data probability distribution of the full training dataset  $D$  in the global model  $\omega_t$  and  $\varepsilon_t$  denotes the total privacy cost at round  $t$ .

**Client selection.** To further reduce storage overhead without compromising recovery performance, we filter client updates via an importance-based mechanism. Specifically, the cosine similarity between a client's update and the global update direction is computed to measure consistency, while the client's remaining privacy budget is incorporated to reflect remaining data utility. These factors are balanced by hyperparameter  $\zeta'$  to derive an importance score, as shown in Eq. 13. Consequently, the server persists only the top  $\lambda'$  fraction of updates. Crucially, this selection is strictly applied to the subset of client updates identified as benign by FIND, thereby limiting storage while mitigating the risk of persisting malicious client updates.

$$X_i = (1 - \zeta') \cdot \mathcal{T}_{cos}(\tilde{g}_{i,t}, \tilde{g}_t) + \zeta' \cdot \varepsilon_{i,t}, \quad (13)$$

$$\mathcal{T}_{cos}(\tilde{g}_{i,t}, \tilde{g}_t) = \langle \tilde{g}_{i,t}, \tilde{g}_t \rangle / (\|\tilde{g}_{i,t}\| \cdot \|\tilde{g}_t\|).$$

$\tilde{g}_t$  is the aggregated update at round  $t$ ,  $\varepsilon_{i,t}$  denotes the remaining privacy budget of client  $i$ , and  $\langle, \rangle$  is the scalar product operator.

#### 4.4 Adaptive Model Rollback

FedRecover [7] and FedEraser [26] recover from the initial global model, which incur high costs. Crab [15] introduces a rollback mechanism based on sensitivity to malicious updates, which poses reliability risks. In contrast, DIARY consolidates detection results of FIND to identify the complete set of malicious clients  $S_m$  and rolls the global model back to a historical checkpoint  $t^*$  that has never aggregated malicious updates. Due to FIND's per-round detection capability, most early preserved checkpoints remain uncontaminated, enabling DIARY to rollback to a relatively recent global

model. This approach reduces recovery overhead while avoiding the risk of restoring from contaminated models.

#### 4.5 Model Recovery

After determining the set of malicious clients  $S_m$  and rollback round  $t^*$ , the server can utilize the selectively stored historical information for model recovery. However, these historical updates cannot be directly applied, as they may have been indirectly contaminated. To address this, we adopt the calibration method from FedEraser [26], which is particularly compatible with our noise injection strategy. Specifically, our recovery method includes four steps: update renewal, update calibration, update aggregation, and model updating.

**Update renewal.** During the  $r$ -th round of the recovery phase, for each true benign client  $i \in S_{b,t}^* \setminus S_m$ , local retraining is performed based on recovered global model  $\hat{\omega}_r$  from the previous recovery round to generate a new client update  $\tilde{u}_{i,r}$ . Here,  $S_{b,t}^*$  denotes the set of stored clients at the round  $t$  in the training phase.

**Update calibration.** Using the renewed update  $\tilde{u}_{i,r}$ , the server can calibrate the corresponding stored historical update  $\tilde{g}_{i,t}$ . Specifically, the norm of  $\tilde{g}_{i,t}$  indicates the magnitude of change required for the global model parameters, while the normalized  $\tilde{u}_{i,r}$  indicates the direction in which the model should be updated, as shown in Eq. 14. More importantly, this approach is highly compatible with our noise injection strategy IPLN: since  $\tilde{g}_{i,t}$  is obtained during the training phase with stronger noise injection, its norm tends to be larger; whereas  $\tilde{u}_{i,r}$  is generated in the recovery phase with milder noise, allowing it to capture the update direction more accurately.

$$\hat{u}_{i,r} \leftarrow \|\tilde{g}_{i,t}\| \cdot \tilde{u}_{i,r} / \|\tilde{u}_{i,r}\|. \quad (14)$$

**Update aggregation.** The server directly applies the aggregation rule  $\mathcal{A}$  (e.g. FedAvg) after collecting all calibrated updates from participating clients:

$$\hat{u}_r = \mathcal{A}(\{\hat{u}_{i,r} | i \in S_{b,t}^* \setminus S_m\}) = (1/|S_{b,t}^* \setminus S_m|) \cdot \sum_{i \in S_{b,t}^* \setminus S_m} \hat{u}_{i,r}. \quad (15)$$

**Model updating.** With the aggregation of the calibrated updates, server can thus update the global model as:

$$\hat{\omega}_{r+1} \leftarrow \hat{\omega}_r + \eta \hat{u}_r. \quad (16)$$

The aforementioned four steps are iteratively executed until all stored historical updates are calibrated and used to update the global model, ultimately yielding the recovered global model.

#### 4.6 Improvement

While selective storage significantly reduces the storage overhead, it introduces uncertainty regarding the exact number of recovery rounds required for each client until the training phase concludes. To strictly preserve privacy guarantees, we initially adopt a conservative estimation of the total participation rounds (including both training and recovery phases) during budget allocation. Consequently, the resulting noise multiplier is relatively high, leading to considerable remaining privacy budget and suboptimal utilization of client data. To address this, we reallocate the privacy budget (e.g. recalculate the noise multiplier) for each client after the training phase concludes, based on the actual number of stored historical updates, which corresponds precisely to the required recovery participation. We term this strategy Double Inverse Power Law Noise (DIPLN) and evaluate its efficacy in Section 6.2.

## 5 Theoretical Analysis

**THEOREM 1.** Given  $(\epsilon, \delta)$ -DP mechanisms of a client at each single iteration  $\tau \in [TE]$ , its global privacy loss after  $TE$  sequential accesses to the local dataset is the least value of  $\epsilon^*$  satisfying that

$$\frac{\sum_{S \subseteq [TE]} \max \{e^{\sum_{\tau \in S} \epsilon_\tau} - e^{\epsilon^* + \sum_{\tau \notin S} \epsilon_\tau}, 0\}}{\prod_{\tau=1}^{TE} (1 + e^{\epsilon_\tau})} \leq 1 - \frac{1 - \delta^*}{(1 - \delta)^{TE}}, \quad (17)$$

where  $\delta$  is the relaxation rate in each single iteration, while  $\delta^*$  is the global relaxation rate.

**PROOF.** According to [17, 30], the output of arbitrary  $(\epsilon, \delta)$ -DP algorithms can be simulated by a randomized algorithm  $\mathcal{M}_{\epsilon, \delta} : \{0, 1\} \rightarrow \{0, 1, 2, 3\}$ , whose mass probability function is defined as

$$\Pr[\mathcal{M}_{\epsilon, \delta}(0)=0]=\delta, \Pr[\mathcal{M}_{\epsilon, \delta}(0)=1]=(1-\delta)e^\epsilon/(1+e^\epsilon), \quad (18)$$

$$\Pr[\mathcal{M}_{\epsilon, \delta}(0)=2]=(1-\delta)/(1+e^\epsilon), \Pr[\mathcal{M}_{\epsilon, \delta}(0)=3]=0, \quad (19)$$

$$\Pr[\mathcal{M}_{\epsilon, \delta}(1)=0]=0, \Pr[\mathcal{M}_{\epsilon, \delta}(1)=1]=(1-\delta)/(1+e^\epsilon), \quad (20)$$

$$\Pr[\mathcal{M}_{\epsilon, \delta}(1)=2]=(1-\delta)e^\epsilon/(1+e^\epsilon), \Pr[\mathcal{M}_{\epsilon, \delta}(1)=3]=\delta. \quad (21)$$

Let  $K=TE$ . The composition of DP mechanisms with parameters  $\{(\epsilon_\tau, \delta) : \tau \in [K]\}$  of the client can be equivalent to the composition

$$\mathcal{M}^K(x) = (\mathcal{M}_{\epsilon_1, \delta}(x), \dots, \mathcal{M}_{\epsilon_\tau, \delta}(x), \dots, \mathcal{M}_{\epsilon_K, \delta}(x)), \quad (22)$$

where  $x \in \{0, 1\}$  and let  $P_x(\cdot)$  denote the mass probability of composition  $\mathcal{M}^K(x)$ . According to the definition of DP as shown in Definition 1, given global relaxation rate  $\delta^*$ , the global privacy loss of  $\mathcal{M}^K(x)$  is the least value of  $\epsilon^*$  such that

$$\delta^* \geq \max_{s \in \{0, 1, 2, 3\}^K} \{P_0(s) - e^{\epsilon^*} P_1(s)\}. \quad (23)$$

To find  $\epsilon^*$  based on Eq. 23, let  $S(\epsilon^*) = \{s \in \{0, 1, 2, 3\}^K : P_0(s) \geq e^{\epsilon^*} P_1(s)\}$  and further partition it into two subsets  $S_1(\epsilon^*), S_2(\epsilon^*)$ :

$$S_1(\epsilon^*) = \{s \in \{0, 1, 2, 3\}^K : P_1(s) = 0\}, \quad (24)$$

$$S_2(\epsilon^*) = \{s \in \{0, 1, 2, 3\}^K : P_1(s) > 0, P_0(s) \geq e^{\epsilon^*} P_1(s)\}. \quad (25)$$

Accordingly,

$$P_0(S(\epsilon^*)) - e^{\epsilon^*} P_1(S(\epsilon^*)) = P_0(S_1(\epsilon^*)) - e^{\epsilon^*} P_1(S_1(\epsilon^*)) + P_0(S_2(\epsilon^*)) - e^{\epsilon^*} P_1(S_2(\epsilon^*)) = 1 - (1 - \delta)^K + P_0(S_2(\epsilon^*)) - e^{\epsilon^*} P_1(S_2(\epsilon^*)), \quad (26)$$

where the first equality is by  $S(\epsilon^*) = S_1(\epsilon^*) \cup S_2(\epsilon^*)$ . The second equality holds by  $P_1(S_1(\epsilon^*)) = 0$  and  $P_0(S_1(\epsilon^*)) = 1 - \Pr[\mathcal{M}^K(0) \in \{1, 2, 3\}^K] = 1 - (1 - \delta)^K$ . The former is due to Eq. 24 and the latter is because when  $P_1(S_1(\epsilon^*)) = 0$ , it means that the output  $s \in S_1(\epsilon^*)$  includes an element of 0 according to Eq. 20. Thus, we evaluate the probability that the output  $s$  includes at least an element of 0 as  $1 - \Pr[\mathcal{M}^K(0) \in \{1, 2, 3\}^K]$ . Furthermore, according to Eq. 25 where  $P_0(s), P_1(s) > 0$ , we have  $S_2(\epsilon^*) \subseteq \{1, 2\}^K$ . Then,

$$\begin{aligned} & P_0(S_2(\epsilon^*)) - e^{\epsilon^*} P_1(S_2(\epsilon^*)) \\ &= \sum_{s \in \{1, 2\}^K} \max \left\{ \prod_{\tau: s_\tau=1} \Pr[\mathcal{M}_{\epsilon, \delta}(0)=1] \prod_{\tau: s_\tau=2} \Pr[\mathcal{M}_{\epsilon, \delta}(0)=2] \right. \\ &\quad \left. - e^{\epsilon^*} \prod_{\tau: s_\tau=1} \Pr[\mathcal{M}_{\epsilon, \delta}(1)=1] \prod_{\tau: s_\tau=2} \Pr[\mathcal{M}_{\epsilon, \delta}(1)=2], 0 \right\}. \end{aligned} \quad (27)$$

By Eqs. 18-21, the global privacy loss  $\epsilon^*$  satisfies that

$$\frac{\sum_{S \subseteq [TE]} \max \{e^{\sum_{\tau \in S} \epsilon_\tau} - e^{\epsilon^* + \sum_{\tau \notin S} \epsilon_\tau}, 0\}}{\prod_{\tau=1}^{TE} (1 + e^{\epsilon_\tau})} \leq 1 - \frac{1 - \delta^*}{(1 - \delta)^{TE}}, \quad (28)$$

which completes the proof.  $\square$

Theorem 1 provides an approach to derive the global privacy loss for the optimal composition of arbitrary DP algorithms over  $TE$

**Table 1: Global privacy loss of various composition methods with number of iterations  $k = 200$ , sample rate  $q = 0.1$ , and noise multiplier  $\sigma = 5$ .**

| Methods                         | $\epsilon$    | $\delta$       |
|---------------------------------|---------------|----------------|
| Basic composition method [8]    | 30.290        | 0.00021        |
| Advanced composition method [8] | 15.231        | 0.00021        |
| RDP composition method [29]     | 17.653        | 0.00021        |
| Algorithm 1                     | <b>10.910</b> | <b>0.00021</b> |

iterations. However, the computational complexity of this solution would be exponential in  $TE$  incurred by computing the sum over subset  $S \subseteq [TE]$ . To solve this concern, we introduce an approximation method described in Algorithm 1.

**THEOREM 2.** Algorithm 1 yields the approximate solution  $\epsilon^*$  of the optimal global privacy loss with complexity of  $O(T^3 E^3 \bar{\epsilon}(1 + \bar{\epsilon}) \log(T^2 E^2 \bar{\epsilon}(1 + \bar{\epsilon})/\xi)/\xi)$ , where  $\bar{\epsilon} = \sum_{\tau \in [TE]} \epsilon_\tau / TE$  and  $\xi \in (0, 1)$ .

**PROOF.** The key to finding the global privacy loss lies in checking the inequality in Eq. 9. To facilitate the check process, we define a new set for computing the summation term on the left side as

$$\begin{aligned} S_0 &= \{S \subseteq [K] : \sum_{\tau \notin S} \hat{\epsilon}_\tau \geq \epsilon^* + \sum_{\tau \in S} \hat{\epsilon}_\tau\} \\ &= \{S \subseteq [K] : \sum_{\tau \in S} a_\tau \leq \lfloor (\sum_{\tau \in S} a_\tau - a^*)/2 \rfloor \}, \end{aligned} \quad (29)$$

where  $K=TE$  and the second inequality is by  $a_\tau = \lceil \epsilon_\tau \cdot (1/\theta + 1) \rceil$  at line 8 in Algorithm 1. The summation term in Eq. 9 can be

$$\begin{aligned} & \sum_{S \subseteq [TE]} \max \{e^{\sum_{\tau \in S} \epsilon_\tau} - e^{\epsilon^* + \sum_{\tau \notin S} \epsilon_\tau}, 0\} \\ &= \sum_{S \subseteq S_0} \left( \prod_{\tau \in S} e^{\epsilon_\tau} \prod_{\tau \in S_0} e^{-\hat{\epsilon}_\tau} - e^{\epsilon^*} \prod_{\tau \in S_0} e^{\hat{\epsilon}_\tau} \right), \end{aligned} \quad (30)$$

where terms  $\prod_{\tau \in S_0} e^{-\epsilon_\tau}$  and  $\prod_{\tau \in S_0} e^{\epsilon_\tau}$  can be efficiently computed like solving knapsack problems based on dynamic programming by setting weight  $w_\tau = e^{-\epsilon_\tau}$  and budget  $B = \lfloor (\sum_{\tau \in S} a_\tau - a^*)/2 \rfloor$ . So the computation complexity of checking the inequality in Eq. 9 is  $O(KB) = O(TE \sum_{\tau \in [TE]} a_\tau) = O(T^3 E^3 \bar{\epsilon}(1 + \bar{\epsilon})/\xi)$  due to line 8 in Algorithm 1. In addition, binary search is utilized to find the value of  $a^*$ , whose complexity is  $O(\log(\sum_{\tau \in [TE]} a_\tau))$ . The complexity of Algorithm 1 is  $O(\frac{T^3 E^3 \bar{\epsilon}(1 + \bar{\epsilon})}{\xi} \log \frac{T^2 E^2 \bar{\epsilon}(1 + \bar{\epsilon})}{\xi})$ .  $\square$

Table 1 depicts the effectiveness of Algorithm 1 in terms of privacy composition compared to existing composition methods.

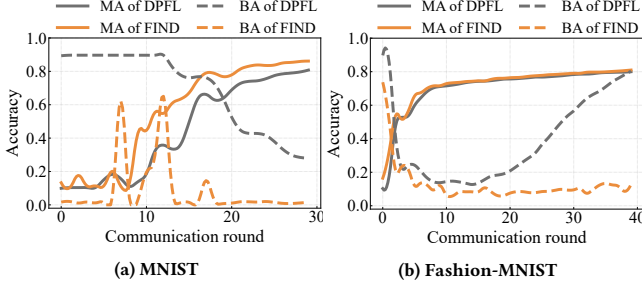
## 6 Numerical Experiments

We use PyTorch Opacus [40] for all experiments and report mean results across five independent runs. For detailed experimental settings, please see Appendix A.1.

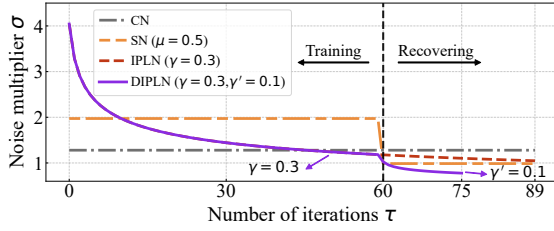
**Datasets and attacks.** We utilize MNIST, Fashion-MNIST, and CIFAR-10 datasets and simulate two attacks: Distributed Backdoor Attack (DBA) [38] and Gradient Inversion Attack (GIA) [39].

**Privacy settings.** We simulate real-world heterogeneity using the following three privacy budget distributions: ThreeLevels, Pareto, and MixGauss.

**Evaluation metrics.** We evaluate the following metrics: (i) Main Accuracy (MA): the model's accuracy on the main task. (ii) Backdoor Accuracy (BA): the model's accuracy on the backdoor task. (iii)



**Figure 4: Test accuracy of FIND under distributed backdoor attacks based on MNIST and Fashion-MNIST datasets.**



**Figure 5: Comparison of various noise injection strategies.**

Remaining privacy budget: reflects data utilization efficiency. (iv) Running time: the duration of the recovery phase. (v) Peak Signal-to-Noise Ratio (PSNR), Structural Similarity (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS): standard metrics for assessing reconstruction quality in gradient inversion attacks.

### 6.1 Evaluation on FIND

As shown in Fig. 4, under the DBA attack, FIND successfully achieves robust defense on the MNIST dataset, with the BA of only 1.56% and MA of 86.83%, outperforming naive differentially private federated learning (DPFL) in both metrics, which attains the BA of 28.03% and the MA of 80.75%. On the Fashion-MNIST dataset, the BA of DPFL reaches 80.29%, while FIND significantly reduces the BA to 12.82% without compromising the MA. In our experiments, FIND achieves a true positive rate of 63.75% and a false positive rate of 12.5%. Although it does not detect all malicious clients, its real-time per-round detection capability can provide support for DIARY.

### 6.2 Evaluation on noise injection strategy

We configure the following three noise injection strategies and one improvement. A detailed illustration of them is provided in Fig. 5.

- **Constant Noise (CN)**: Applies a fixed noise multiplier throughout all iterations.
- **Step Noise (SN)**: Applies a fixed  $\sigma_0$  during the training phase, and switches to a scaled multiplier  $\mu * \sigma_0$  for the recovery phase.
- **Inverse Power Law Noise (IPLN)**: Decays the noise multiplier dynamically according to  $\sigma_\tau = \sigma_0 / (1 + \tau)^\gamma$ .
- **Double Inverse Power Law Noise (DIPLN)**: An improvement over IPLN that reallocates the remaining privacy budget and adopts a new decay rate  $\gamma'$  during the recovery phase, as described in Section 4.6.

As shown in Fig. 7, IPLN and DIPLN achieves the highest MA, followed by SN, while CN performs the worst. This validates the hypothesis that early optimization phases tolerate larger gradients,

**Table 2: Quality of reconstructed data of FedAvg and DIARY under gradient inversion attacks.**

| Methods | Fashion-MNIST |        |         | Cifar10 |        |         |
|---------|---------------|--------|---------|---------|--------|---------|
|         | PSNR ↓        | SSIM ↓ | LPIPS ↑ | PSNR ↓  | SSIM ↓ | LPIPS ↑ |
| FedAvg  | 19.12         | 0.8053 | 0.03    | 18.54   | 0.7245 | 0.06    |
| DIARY   | 5.60          | 0.2348 | 0.67    | 8.41    | 0.1531 | 0.37    |

whereas converging parameters require precise updates. Therefore, allocating more privacy budget to the later phase proves to be a superior strategy that also aligns well with DIARY’s update calibration. Furthermore, DIPLN improves upon IPLN by reallocating the remaining privacy budget during the recovery phase to fully exploit client data utility. This efficiency is evidenced by the minimal remaining privacy budget shown in Fig. 9. Consequently, DIPLN outperforms other strategies, effectively eliminating the backdoor attack with BA of 2.97% while achieving the highest MA of 78.52%. The results on CIFAR-10 are provided in the Appendix A.2.

### 6.3 Evaluation on DIARY

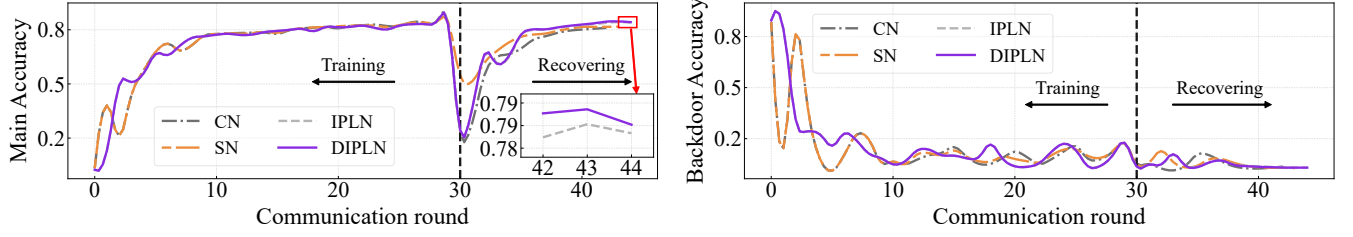
In this section, we compare DIARY with the following representative recovery methods to demonstrate its effectiveness.

- **DPFL**: A baseline method without any detection or recovery.
- **Retrain**: Retrains the global model from scratch using only benign clients after identifying malicious clients.
- **FedRecover** [7]: Offloads update renewal tasks to the server via estimation and correction.
- **FedEraser** [26]: Stores historical information at fixed interval and calibrates them using benign clients for recovery.
- **Crab** [15]: Optimizes storage via importance-based selection of rounds and clients, and design a rollback mechanism.
- **FedADP** [16]: Similar to Crab but omits model rollback and allocates privacy budget based exclusively on training loss.

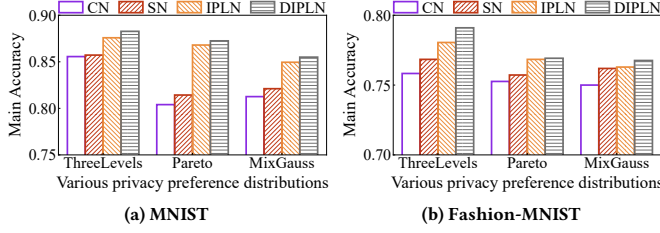
**Performance of DIARY.** As shown in Fig. 8, across both MNIST and Fashion-MNIST datasets, all methods effectively eliminate the impact of backdoor attack. However, DIARY achieves MA closest to that of Retrain, significantly outperforming FedEraser and Crab. This superior performance stems from FIND-driven client selection and rollback mechanism, coupled with adaptive privacy budget allocation, making DIARY uniquely tailored for practical differentially private federated unlearning. Additional results on the CIFAR-10 dataset, along with an evaluation of DIARY’s resilience to varying backdoor attack intensities, are provided in the Appendix A.2.

Furthermore, Table 2 presents the quality of reconstructed data under gradient inversion attacks. The results indicate that FedAvg is highly susceptible to privacy leakage, as reflected by its high PSNR and SSIM scores and low LPIPS value. In contrast, DIARY significantly safeguards client data privacy, causing a substantial deterioration in reconstruction quality. In addition, we provide several visualization results under gradient inversion attacks in Appendix A.2 based on both Fashion-MNIST and Cifar-10 datasets to further demonstrate DIARY’s effectiveness.

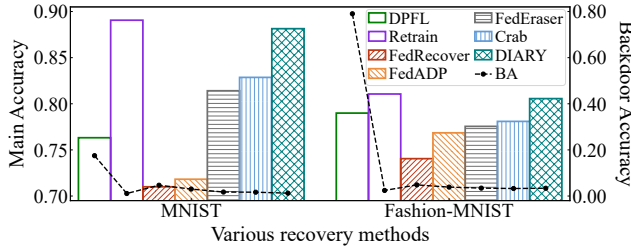
**Memory and computational costs.** To support the recovery phase, some methods require the server to store specific information during the training phase, which introduces additional memory consumption. As for computational cost, we only consider the number of rounds in which clients need to recompute model updates



**Figure 6: Main accuracy and backdoor accuracy of employing various noise injection strategies during model training and recovering phases on Fashion-MNIST.**



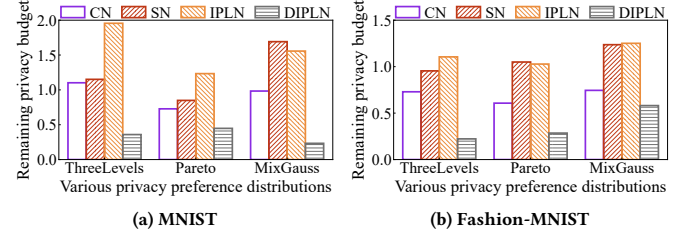
**Figure 7: Main accuracy of various noise injection strategies under different privacy preference distributions.**



**Figure 8: Main accuracy and backdoor accuracy of various recovery methods on MNIST and Fashion-MNIST.**

during the recovery phase. As shown in Table 3, we compare the storage and computational costs of various recovery methods. We assume the model has  $M$  parameters, the total number of clients is  $N$ , and the total training rounds is  $T$ . Retrain does not require storing any information during training phase, but it incurs the highest computational cost, which is  $O(T)$ . FedRecover requires server to store each round information from all clients, resulting in the highest storage cost of  $O(NMT)$ , and its computational cost is  $O\left(T_w + T_f + \lfloor \frac{(T-T_w-T_f)}{T_c} \rfloor\right)$ , relying on the warm-up rounds  $T_w$ , the periodic correction  $T_c$  and final tuning rounds  $T_f$ .

FedEraser stores information every  $\Delta t$  rounds, with a storage cost of  $O(NM\lfloor \frac{T}{\Delta t} \rfloor)$  and a computational cost of  $O(\lfloor \frac{T}{\Delta t} \rfloor)$ . In Crab, the cost depends on the selective stored rounds and clients and rollback round. Therefore, the storage cost and computation cost can be  $O(\lambda\lambda'NMT)$  and  $O(\lambda T - j^*)$ , where  $j^*$  is the index of rollback round in stored rounds. FedADP is similar to Crab, but it lacks model rollback, so its computation cost is  $O(\lambda T)$ . DIARY is also similar to Crab in strategy, but selects clients based on FIND's detection results, resulting in a lower storage cost of  $O(\phi\lambda\lambda'NMT)$  and a computation cost of  $O(\lambda T - j^*)$ , where  $\phi$  denotes the proportion of benign clients identified by FIND. Additionally, we evaluate the running time of various recovery methods on multiple datasets in Appendix A.2, further demonstrating DIARY's efficiency.



**Figure 9: Remaining privacy budgets of various noise injection strategies under different preference distributions.**

**Table 3: Memory and computation costs of various recovery methods.**

| Methods    | Memory cost                               | Computation cost                                         |
|------------|-------------------------------------------|----------------------------------------------------------|
| Retrain    | None                                      | $O(T)$                                                   |
| FedRecover | $O(NMT)$                                  | $O(T_w + T_f + \lfloor \frac{(T-T_w-T_f)}{T_c} \rfloor)$ |
| FedEraser  | $O(NM\lfloor \frac{T}{\Delta t} \rfloor)$ | $O(\lfloor \frac{T}{\Delta t} \rfloor)$                  |
| FedADP     | $O(\lambda\lambda'NMT)$                   | $O(\lambda T)$                                           |
| Crab       | $O(\lambda\lambda'NMT)$                   | $O(\lambda T - j^*)$                                     |
| DIARY      | $O(\phi\lambda\lambda'NMT)$               | $O(\lambda T - j^*)$                                     |

## 7 Conclusion

In this paper, we primarily focus on a critical issue in existing federated unlearning studies: the risk of privacy leakage from client data. To solve this problem, we integrate differential privacy into federated unlearning and propose an adaptive privacy budget allocation mechanism alongside a novel method named DIARY. Through theoretical analysis and extensive experiments, we demonstrate that DIARY effectively and efficiently completes federated unlearning while rigorously protecting the data privacy of clients.

DIARY's key insight for balancing the privacy-utility trade-off lies in adaptively injecting appropriate noise in various phases to fully leverage clients' global privacy budget. This approach extends beyond federated unlearning, offering great potential for diverse privacy-sensitive applications.

## Acknowledgments

This work has been partially supported by National Natural Science Foundation of China under Grant No. 62302314, Shenzhen Science and Technology Program under Grant No. RCBS20231211090647075, National Key Research and Development Program under Grant No. 2024YFB3310200, the Key R&D plan of Jilin Province under Grant No. 20250201076GX, and Scientific Foundation for Youth Scholars of Shenzhen University under Grant No. 8270001024.

## References

- [1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *Proc. of CCS*. 308–318.
- [2] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. 2020. How to backdoor federated learning. In *Proc. of AISTATS*. 2938–2948.
- [3] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. 2017. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Proc. of NeurIPS*, Vol. 30.
- [4] Lucas Bourtole, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *Proc. of IEEE S&P*. 141–159.
- [5] Jianping Cai, Ximeng Liu, Qingqing Ye, Yang Liu, and Yuyang Wang. 2024. A federated learning framework based on differentially private continuous data release. *IEEE Transactions on Dependable and Secure Computing* 21, 5 (2024), 4879–4894.
- [6] Ricardo JGB Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-based clustering based on hierarchical density estimates. In *Proc. of PAKDD*. 160–172.
- [7] Xiaoyu Cao, Jinyuan Jia, Zaixi Zhang, and Neil Zhenqiang Gong. 2023. Fedrecover: Recovering from poisoning attacks in federated learning using historical information. In *Proc. of IEEE S&P*. 1366–1383.
- [8] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and trends® in theoretical computer science* 9, 3–4 (2014), 211–407.
- [9] D European Union. 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). *Official Journal of the European Union* 119, 1 (2016), 1–88.
- [10] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. 2020. Inverting gradients - how easy is it to break privacy in federated learning?. In *Proc. of NeurIPS*. 16937–16947.
- [11] Rachid Guerraoui, Sébastien Rouault, et al. 2018. The hidden vulnerability of distributed learning in byzantium. In *Proc. of ICML*. 3521–3530.
- [12] Anisa Halimi, Swanand Ravindra Kadhe, Amrith Rawat, and Nathalie Baracaldo Angel. 2022. Federated Unlearning: How to Efficiently Erase a Client in FL? *arXiv preprint arXiv:2207.05521* (2022).
- [13] Elizabeth Liz Harding, Jarno J Vanto, Reece Clark, L Hannah Ji, and Sara C Ainsworth. 2019. Understanding the scope and impact of the california consumer privacy act of 2018. *Journal of Data Protection & Privacy* 2, 3 (2019), 234–253.
- [14] Yuke Hu, Wei Liang, Ruofan Wu, Kai Xiao, Weiqiang Wang, Xiaochen Li, Jinfei Liu, and Zhan Qin. 2023. Quantifying and defending against privacy threats on federated knowledge graph embedding. In *Proc. of WWW*. 2306–2317.
- [15] Yu Jiang, Jiyuan Shen, Ziyao Liu, Chee Wei Tan, and Kwok-Yan Lam. 2025. Towards efficient and certified recovery from poisoning attacks in federated learning. *IEEE Transactions on Information Forensics and Security* 20 (2025), 2632–2647.
- [16] Yu Jiang, Xindi Tong, Ziyao Liu, Huanyu Ye, Chee Wei Tan, and Kwok-Yan Lam. 2024. Efficient federated unlearning with adaptive differential privacy preservation. In *Proc. of IEEE BigData*. 7822–7831.
- [17] Peter Kairouz, Sewoong Oh, and Pramod Viswanath. 2017. The Composition Theorem for Differential Privacy. *IEEE Transactions on Information Theory* 63, 6 (2017), 4037–4049.
- [18] Kris Klein. 2020. *Canadian privacy: Data protection and policy for the practitioner*. International Association of Privacy Professionals.
- [19] Alex Krizhevsky and Geoffrey Hinton. 2009. Learning multiple layers of features from tiny images. (2009), 32–33.
- [20] Yann LeCun. 1998. The MNIST database of handwritten digits. <http://yann.lecun.com/exdb/mnist/> (1998).
- [21] Jaewoo Lee and Daniel Kifer. 2018. Concentrated differentially private gradient descent with adaptive per-iteration privacy budget. In *Proc. of KDD*. 1656–1665.
- [22] Ke Liang, Yue Liu, Sihang Zhou, Wenxuan Tu, Yi Wen, Xihong Yang, Xiangjun Dong, and Xinwang Liu. 2023. Knowledge graph contrastive learning based on relation-symmetrical structure. *IEEE Transactions on Knowledge and Data Engineering* 36, 1 (2023), 226–238.
- [23] Ke Liang, Lingyuan Meng, Hao Li, Jun Wang, Long Lan, Miaomiao Li, Xinwang Liu, and Huaimin Wang. 2025. From concrete to abstract: multi-view clustering on relational knowledge. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47, 10 (2025), 9043–9060.
- [24] Ke Liang, Lingyuan Meng, Meng Liu, Yue Liu, Wenxuan Tu, Siwei Wang, Sihang Zhou, and Xinwang Liu. 2023. Learn from relational correlations and periodic events for temporal knowledge graph reasoning. In *Proc. of SIGIR*. 1559–1568.
- [25] Yijing Lin, Zhipeng Gao, Hongyang Du, Dusit Niyato, Gui Gui, Shuguang Cui, and Jinke Ren. 2024. Scalable federated unlearning via isolated and coded sharding. In *Proc. of IJCAI*. 4551–4559.
- [26] Gaoyang Liu, Xiaoqiang Ma, Yang Yang, Chen Wang, and Jiangchuan Liu. 2021. Federaser: Enabling efficient client-level data removal from federated learning models. In *Proc. of IEEE/ACM IWQoS*. 1–10.
- [27] Junxu Liu, Jian Lou, Li Xiong, Jinfei Liu, and Xiaofeng Meng. 2024. Cross-silo federated learning with record-level personalized differential privacy. In *Proc. of CCS*. 303–317.
- [28] Ziyao Liu, Yu Jiang, Weifeng Jiang, Jiale Guo, Jun Zhao, and Kwok-Yan Lam. 2024. Guaranteeing data privacy in federated unlearning with dynamic user participation. *IEEE Transactions on Dependable and Secure Computing* 22, 3 (2024), 2072–2085.
- [29] Ilya Mironov. 2017. Rényi differential privacy. In *Proc. of IEEE CSF*. 263–275.
- [30] Jack Murtagh and Salil Vadhan. 2015. The complexity of computing the optimal composition of differential privacy. In *Proc. of TCC*. 157–175.
- [31] Thien Duc Nguyen, Phillip Rieger, Huili Chen, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, Azalia Mirhoseini, and Shaza Zeitouni. 2022. FLAME: Taming backdoors in federated learning. In *Proc. of USENIX Security*. 1415–1432.
- [32] Tao Qi, Huili Wang, and Yongfeng Huang. 2024. Towards the robustness of differentially private federated learning. In *Proc. of AAAI*, Vol. 38. 19911–19919.
- [33] Hongyu Qiu, Yongwei Wang, Yonghui Xu, Lizhen Cui, and Zhiqi Shen. 2023. Fedcio: Efficient exact federated unlearning with clustering, isolation, and one-shot aggregation. In *Proc. of IEEE BigData*. 5559–5568.
- [34] Youming Tao, Cheng-Long Wang, Miao Pan, Dongxiao Yu, Xiuzhen Cheng, and Di Wang. 2024. Communication Efficient and Provable Federated Unlearning. *Proceedings of the Very Large Data Bases Endowment* 17, 5 (2024), 1119–1131.
- [35] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A practical guide, 1st ed.*, Cham: Springer International Publishing 10, 3152676 (2017), 10–5555.
- [36] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H Yang, Farhad Farokhi, Shi Jin, Tony QS Quek, and H Vincent Poor. 2020. Federated learning with differential privacy: algorithms and performance analysis. *IEEE Transactions on information forensics and security* 15 (2020), 3454–3469.
- [37] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747* (2017).
- [38] Chulin Xie, Keli Huang, Pin-Yu Chen, and Bo Li. 2020. DBA: Distributed Backdoor Attacks against Federated Learning. In *Proc. of ICLR*.
- [39] Zipeng Ye, Wenjian Luo, Qi Zhou, and Yubo Tang. 2024. High-fidelity gradient inversion in distributed learning. In *Proc. of AAAI*, Vol. 38. 19983–19991.
- [40] Ashkan Yousefpour, Igor Shilov, Alexandre Sablayrolles, Davide Testuggine, Karthik Prasad, Mani Malek, John Nguyen, Sayan Ghosh, Akash Bharadwaj, Jessica Zhao, Graham Cormode, and Ilya Mironov. 2021. Opacus: user-friendly differential privacy library in PyTorch. In *Proc. of NeurIPS Workshop Privacy in Machine Learning*.
- [41] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. In *Proc. of NeurIPS*, Vol. 32.

## A Appendix

### A.1 Detailed Experimental Settings

**Datasets and models.** The detailed model architectures for each dataset are presented in Table 4.

**Parameters.** The following hyperparameter values are configured to facilitate a comparative analysis of the experimental results. Unless otherwise specified, these parameters remain consistent across all three datasets. We set the round selection ratio  $\lambda$  to 0.5 and client selection ratio  $\lambda'$  to 0.8. Privacy cost ratio  $\zeta, \zeta'$  and percentage of reduction  $\beta, \beta'$  are all set to 0.2. The DP relaxation rate  $\delta$  is set to 0.001. The parameters of Algorithm 1:  $\delta^* = 0.1, \xi = 0.5$ . The remaining hyperparameter settings are in Table 5.

**Attack settings.** We simulate the Distributed Backdoor Attack (DBA) [38] and Gradient Inversion Attack (GIA) [39].

**Table 4: Description of used datasets and models.**

| Datasets           | Model Architecture                                             |
|--------------------|----------------------------------------------------------------|
| MNIST [20]         | 2 Conv., 2 Pool., and 2 FC layers                              |
| FASHION-MNIST [37] | 3 Conv. (with GroupNorm), 2 Pool., and 1 FC layers             |
| CIFAR-10 [19]      | 2 Conv. (with GroupNorm), 2 Pool., 1 Dropout., and 2 FC layers |

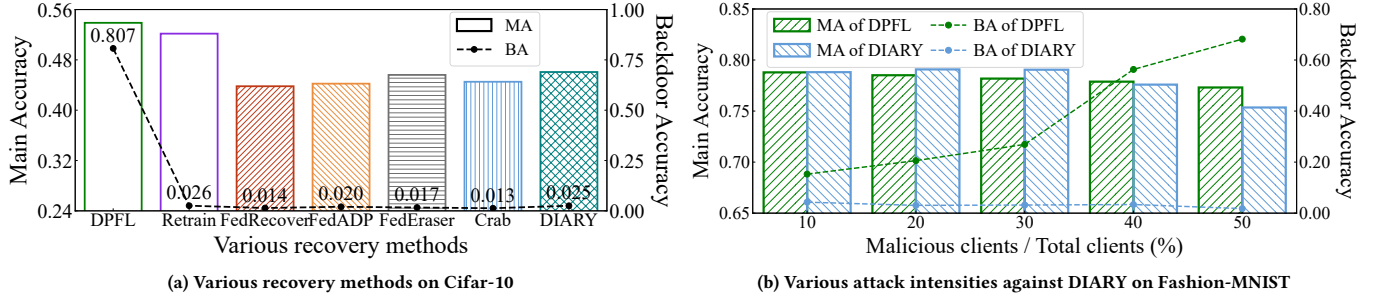


Figure 10: Main accuracy (MA) and backdoor accuracy (BA) under backdoor attacks.



Figure 11: Illustration of gradient inversion attack results based on Fashion-MNIST (left part) and Cifar-10 (right part).

Table 5: Hyperparameter configurations for various datasets.

| Hyperparameter           | MNIST | Fashion-MNIST | CIFAR-10 |
|--------------------------|-------|---------------|----------|
| Training round $T$       | 30    | 40            | 150      |
| Recovery round $T'$      | 20    | 30            | 100      |
| Training step $E$        | 2     | 2             | 2        |
| Recovery step $E'$       | 2     | 2             | 2        |
| Clients $N$              | 10    | 10            | 10       |
| Learning rate $\eta$     | 0.05  | 0.05          | 0.1      |
| Data sampling rate $q$   | 0.25  | 0.25          | 0.25     |
| Decay rate $\gamma$      | 0.6   | 0.5           | 0.3      |
| New decay rate $\gamma'$ | 0.1   | 0.1           | 0.05     |

Table 6: Running time (seconds) of various recovery methods.

| Methods    | MNIST | Fashion-MNIST | CIFAR-10 |
|------------|-------|---------------|----------|
| Retrain    | 25    | 154           | 5484     |
| FedRecover | 35    | 86            | 2232     |
| FedEraser  | 22    | 70            | 1216     |
| FedAPD     | 21    | 56            | 801      |
| Crab       | 14    | 40            | 561      |
| DIARY      | 18    | 42            | 603      |

- **Distributed backdoor attack.** We decompose a global trigger pattern “==” into separate local patterns “-” assigned to malicious clients to minimize gradient anomalies. By default, 4 out of 10 clients are selected malicious, targeting label 2 in every training round.
- **Gradient inversion attack.** We simulate the gradient inversion attack on Cifar-10 and Fashion-MNIST datasets, with image resolutions of  $28 \times 28$  and  $32 \times 32$ , respectively. Without loss of generality, the batch size is set to 4.

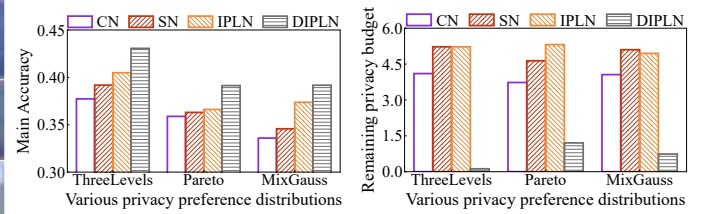


Figure 12: Main accuracy (left part) and remaining privacy budget (right part) of various noise injection strategies under different privacy preference distributions on Cifar-10.

**Privacy preference distributions.** We then simulate real-world heterogeneity using three privacy budget distributions:

- **ThreeLevels:** Each client selects a privacy budget from three predefined options (strong, moderate or weak protection) simulating real-world personalized differentially private implementation with discrete users’ choices.
- **Pareto:** Clients’ budgets follow an approximate Pareto distribution, simulating a population where most prefer strong privacy, while a minority accepts weaker protection.
- **MixGauss:** Clients’ budgets follow a distribution that approximates a mixture of three Gaussian distributions, simulating multimodal preference distributions commonly in social systems.

**Implementation.** All experiments are conducted on a work station equipped with an NVIDIA GeForce RTX 5060 GPU (8GB) and an Intel i7-12700KF processor with 32GB of DDR5 RAM.

## A.2 Additional Experimental Results

**Evaluation on DIARY against various attacks.** Fig. 10a presents the performance of various recovery methods on Cifar-10. The empirical results demonstrate that DIARY effectively eliminates the impact of backdoor attack, reducing the Backdoor Accuracy to a mere 2.5%. In contrast, DPFL exhibits the Backdoor Accuracy of 80.7%, indicating that it remains compromised by the backdoor attack. Furthermore, in terms of Main Accuracy, DIARY outperforms all other recovery methods. While there remains a slight performance gap compared to Retrain, DIARY compensates for this by significantly reducing the recovery overhead. GIA results are illustrated in Fig. 11, where the top row displays the ground truth images, while the second and third rows show reconstructed images under FedAvg and DIARY, respectively. We can observe that FedAvg has a much higher risk of data information leakage, as

**Algorithm 3:** Selective Storage

---

**Input:** The  $k$ -th time window  $W_k = \{t', t'+1, \dots, t'+L_k-1\}$  with a size of  $L_k$  and  $t' = \sum_{j=1}^{k-1} L_j$ , which stored global models  $S_{\omega,k} = \{\omega_t | t \in W_k\} \cup \{\omega_{t'+L_k}\}$ , clients  $\{S_{b,t} | t \in W_k\}$  with corresponding updates and remaining privacy budgets; round selection ratio  $\lambda$ , client selection ratio  $\lambda'$ , round privacy cost ratio  $\zeta$  and client privacy budget ratio  $\zeta'$

**Output:** Selected global model  $S_{\omega,k}^*$  with corresponding clients  $\{S_{b,t}^* | t \in W_k^*\}$

---

```

1 Initial empty sets  $\mathcal{S}, \mathcal{V}$ ;
  // Round selection
2 foreach  $t \in W_k$  do
3    $\varepsilon_t = \sum_{i \in S_{b,t}} (\varepsilon_{i,t} - \varepsilon_{i,t+1})$ ;
4    $P_t = (1 - \zeta) \cdot \mathcal{D}_{kl}(\omega_t, \omega_{t+1}) + \zeta \cdot \varepsilon_t$ ;  $\mathcal{S} \leftarrow \mathcal{S} \cup \{P_t\}$ ;
5  $W_k^* \leftarrow$  Select top  $\lambda L_k$  rounds out of  $W_k$  based on  $\mathcal{S}$ ;
6  $S_{\omega,k}^* \leftarrow \{\omega_t | t \in W_k^*\}$ ;
  // Client selection
7 foreach  $t \in W_k^*$  do
8    $\tilde{g}_t \leftarrow \mathcal{A}(\{\tilde{g}_{i,t} | i \in S_{b,t}\})$ ;
9   foreach  $i \in S_{b,t}$  do
10     $X_i = (1 - \zeta') \cdot \mathcal{T}_{cos}(\tilde{g}_{i,t}, \tilde{g}_t) + \zeta' \cdot \varepsilon_{i,t}$ ;
11     $\mathcal{V} \leftarrow \mathcal{V} \cup \{X_i\}$ ;
12  $S_{b,t}^* \leftarrow$  Select top  $\lambda' |S_{b,t}|$  clients out of  $S_{b,t}$  based on  $\mathcal{V}$ ;
13  $\mathcal{V} \leftarrow \emptyset$ ;
14 return  $S_{\omega,k}^*, \{S_{b,t}^* | t \in W_k^*\}$ 

```

---

indicated by the reconstructed images closely resembling the original ones. In contrast, DIARY successfully masks sensitive features and prevents the extraction of private training data.

**Impact of backdoor attack intensity.** As shown in Fig. 10b, we evaluate the recovery performance of DIARY under varying

backdoor attack intensities, i.e., different proportions of malicious clients. Under all settings, DIARY successfully eliminates the impact of backdoor attack, with the Backdoor Accuracy consistently remaining below 5%. When the proportion of malicious clients ranges from 10% to 40%, DIARY maintains a high main accuracy without significant degradation, and in some cases even outperforms the DPFL. However, under the extreme scenario where the malicious client ratio reaches 50%, the Main Accuracy of DIARY is 75.35%, slightly lower than 77.31% of DPFL. Overall, DIARY demonstrated strong robustness against varying proportions of malicious clients, except under extreme conditions.

**Evaluation on noise injection strategies.** Fig. 12 presents the Main Accuracy and remaining privacy budgets of various noise injection strategy under different privacy preference distributions on Cifar-10. The empirical results demonstrate that both IPLN and DIPLN outperform CN and SN in terms of Main Accuracy, reaffirming the superiority of our privacy budget allocation method in facilitating model training. Furthermore, DIPLN achieves a higher Main Accuracy than IPLN. Notably, while IPLN leaves a substantial remaining privacy budget, DIPLN's remaining privacy budget approaches zero. This validates the effectiveness of DIPLN's strategy on reallocating the remaining privacy budget during the recovery phase, making a significant improvement over IPLN.

**Evaluation on running time.** As shown in Table 6, we evaluate the running time of various recovery methods based on several datasets. It can be observed that FedRecover, which employs the L-BFGS algorithm for Hessian matrix calculation, incurs significant computational overhead. In contrast, due to its two-level selection strategy and model rollback mechanisms, Crab achieves lower running time compared to FedEraser and FedADP. While DIARY is comparable to Crab, it incorporates privacy budget constraints into its selection and employs a more conservative yet safer model rollback mechanism. Consequently, DIARY achieves a modest increase in running time.

**Algorithm 4:** DIARY

---

**Input:** Initial global model  $\omega_0$ , client  $i \in [N]$  with dataset  $D_i$  and privacy budget  $\epsilon_{i,0}$ , total number of rounds in training phase  $T$ , number of local training steps in training phase  $E$ , estimated total number of rounds in recovery phase  $T'$ , estimated number of local training steps in recovery phase  $E'$ , Percentage of loss reduction  $\beta$ , Percentage of privacy budget reduction  $\beta'$ , The DP relaxation rate  $\delta$ , The data sampling rate  $q$ , The decay rates  $\gamma, \gamma'$

**Output:** Recovered global model  $\hat{\omega}_r$

**// First Privacy Budgets Allocation**

1 **for**  $i \leftarrow 0$  **to**  $N - 1$  **do**

2    $\sigma_{i,0} \leftarrow$  find initial noise multiplier such that  $\epsilon_{i,0} \approx \epsilon^*$ , by using  $\text{PCE}(\epsilon_{i,0}, TE + \hat{T}\hat{E}, \sigma_{i,0}, q, \gamma)$  combined with binary search;

**// Training Phase:**

3 Initial  $\ell \leftarrow F(\omega_0)$ ,  $S_{\omega,k} \leftarrow \{\omega_0\}$ ,  $W^* \leftarrow \emptyset$ ,  $k \leftarrow 1$ ,  $r \leftarrow 0$ ,  $\hat{h} \leftarrow 0$ ;

4 **for**  $t \leftarrow 0$  **to**  $T - 1$  **do**

5   **for**  $i \leftarrow 0$  **to**  $N - 1$  **in parallel do**

6      $\tilde{g}_{i,t}, \epsilon_{i,t+1} \leftarrow$  client  $i$  performs local training based on  $\omega_t, D_i, \sigma_{i,0}$ ;

7   The set of benign clients  $S_{b,t}$ , The set of malicious clients  $S_{m,t} \leftarrow \text{FIND}(\tilde{g}_{0,t}, \tilde{g}_{1,t}, \dots, \tilde{g}_{N-1,t})$ ;

8   **if**  $t = 0$  **then**

9      $\hat{h} \leftarrow \sum_{i \in S_{b,t}} \epsilon_{i,t}$ ;

10   The server aggregates the updates  $\tilde{g}_t \leftarrow \mathcal{A}(\{\tilde{g}_{i,t} | i \in S_{b,t}\})$ ;

11   The server updates the global model  $\omega_{t+1} \leftarrow \omega_t + \eta \tilde{g}_t$ ;

12   The server temporarily stores the set of global model  $S_{\omega,k} \leftarrow S_{\omega,k} \cup \{\omega_{t+1}\}$ , the set of clients  $S_{b,t}$  with corresponding updates and remaining privacy budgets in this round;

13   **if**  $F(\omega_{t+1}) \leq (1 - \beta) \cdot \ell$  **or**  $\sum_{i \in S_{b,t}} \epsilon_{i,t+1} \leq (1 - \beta') \cdot \hat{h}$  **then**

14     The server computes the size  $L_k$  for the  $k$ -th time window  $W_k$ ;

15     The server stores  $S_{\omega,k}^*, \{S_{b,t}^* | t \in W_k\} \leftarrow \text{SelectiveStorage}(W_k, S_{\omega,k}, \{S_{b,t} | t \in W_k\})$ ;

16      $W^* \leftarrow W^* \cup W_k^*$ ;

17      $k \leftarrow k + 1$ ;

18      $\ell \leftarrow F(\omega_{t+1})$ ;

19      $\hat{h} \leftarrow \sum_{i \in S_{b,t}} \epsilon_{i,t+1}$ ;

20 At the end of  $T$ , The server finally stores selected global models  $S_{\omega}^* = \{\omega_t | t \in W^*\}$  with corresponding clients  $\{S_{b,t}^* | t \in W^*\}$  and determines the final confirmed set of malicious clients  $S_m$  based on  $\{S_{m,t} | t \in [T]\}$ ;

**// Recovery Phase:**

21 The server determines the roll back round  $t^*$  that stored latest global model  $\omega_{t^*}$  unaffected by malicious clients' updates;

22  $W^* \leftarrow \{t | t \in W^* \text{ and } t \geq t^*\}$ ;

**// Second Privacy Budgets Allocation**

23 **foreach**  $i \in \{S_{b,t}^* | t \in W^*\}$  **do**

24    $\sigma'_{i,0} \leftarrow$  find new initial noise multiplier such that  $\epsilon_{i,0} \approx \epsilon^*$ , by using  $\text{PCE}(\epsilon_{i,0}, TE + T'_i E'_i, \sigma'_{i,0}, q, \gamma')$ , which should composite the historical privacy costs, combined with binary search, And  $T'_i$  and  $E'_i$  denote the true number of training rounds and local steps required for client  $i$  in recovery phase;

25  $\hat{\omega}_0 \leftarrow \omega_{t^*}$ ;

26 **foreach**  $t \in W^*$  **and**  $t \geq t^*$  **do**

27   **foreach**  $i \in S_{b,t}^* \setminus S_m$  **in parallel do**

28      $\tilde{u}_{i,r} \leftarrow$  client  $i$  performs local training based on  $\hat{\omega}_r, D_i, \sigma'_{i,0}$ ;

29     The server calibrates the update  $\hat{u}_{i,r} \leftarrow \|\tilde{g}_{i,t}\| \cdot \frac{\tilde{u}_{i,r}}{\|\tilde{u}_{i,r}\|}$ ;

30   The server aggregates the updates  $\hat{u}_r \leftarrow \mathcal{A}(\{\hat{u}_{i,r} | i \in S_{b,t}^* \setminus S_m\})$ ;

31   The server updates the global model  $\hat{\omega}_{r+1} \leftarrow \hat{\omega}_r + \eta \hat{u}_r$ ;

32    $r \leftarrow r + 1$ ;

33 **return**  $\hat{\omega}_r$

---